ON THIS PAGE
01 Welcome
02 How it works
03 Pick a repo
04 Problem Description
05 Tests
06 Solution
07 Dockerfile
08 Submitting
09 Local Review
10 Criteria
11 Example
12 Tips

Creating Olympus Challenges

01

Welcome to Quest Olympus

Expect to read this doc multiple times until you get your first one or two approvals. I really recommend having it open in a separate tab by your side.

Submissions in Olympus are basically you: creating a task similar to a GitHub issue, writing its tests, and the solution that will pass all of these new tests.

> Note that the required submissions have to be **really challenging** to challenge SOTA models. If you think you know what "challenging" means, you'll probably need to bump it up a notch — you'll be surprised.

This tab covers the submission itself. Its sibling, **The Process**, covers everything you'll run against it: the checks, the agent rollouts, and review. Read this one first, then that one before you spend tokens.

02

How a submission works

To create a submission you'll need:

01 **A repo and a commit hash** — the codebase your task lives in, pinned to a single SHA.
02 **A problem description** — the task itself.
03 **Tests** — what defines success.
04 **A solution** — the proof your task is solvable.
05 **A Dockerfile** — the environment everything runs in.

The rest of this doc walks each one in detail, then ends with a complete approved example you can see in full.

03

Pick a repo

Find a nice GitHub repo that meets the requirements below. This is the foundation. Everything downstream assumes a serious, active project.

Requirements

- Public GitHub repository
- At least 1 commit in the last 12 months
- 500+ stars
- Production-level codebase
- Language: TypeScript, JavaScript, Python, Go, Rust, C++, or Java
- Permissive open-source license (see allowed list below)
- Optional: a GitHub issue URL that describes the problem

▶ Allowed licenses

What makes a repo good

The requirements above are the floor, not the goal. What actually decides how far you can go is the repo's depth. A thin or flat repo caps your difficulty ceiling no matter how good your idea is. Some repos just make much better tasks than others, and it's worth spending real time on this choice before committing.

Take care

R1 Don't use inactive or abandoned repositories.
R2 **No existing PR solves it**: open, merged, or closed. A closed or unmerged PR that already implements your idea still rules it out; not landing doesn't make it fresh. (Saying it again, it's the #1 rejection reason.)
R3 **The maintainers haven't declined it** — check the issues and the GitHub Discussions, not just PRs; that's where design rulings tend to live. A declined feature counts as misalignment with the repo.
R4 Don't invent nonsensical features that don't fit the project's philosophy. Read the readme and get a feel for the context before you commit to an idea.

04

The problem description (aka the task)

P1 Aligns with the repo's philosophy
P2 Not already fixed in an open or merged PR (I know, duplicate point. It is really important)
P3 **Self-contained** — solvable from the repo and description alone
P4 **Clear, concise, and unambiguous** — describes what to build or fix and don't leave points for guessing
P5 **Verifiable** — success is objectively testable
P6 **Not prescriptive** — don't leak the solution, we want to challenge the agents, remember!!
P7 **Not a duplicate** — we run a similarity check against existing submissions. Don't treat it as a gate; open the results, read the close matches it surfaces, and confirm you're not rebuilding a problem that already exists. Rewording the same task or reshaping the same behavior doesn't make it new.

Writing it

A note on how it should read: write it the way a maintainer writes an issue — **natural prose, full sentences**. Open with the ask itself ("Add X to Y", "Fix Z when …") and let the first line stand on its own without the title. Skip the motivation and the "what the repo currently lacks" preamble.

Keep it out of spec-sheet territory: no bulleted requirement lists, no headings, no code snippets doing the describing. And don't spell out what a developer in the repo would find on their own (internal class names, helpers, field names, the file layout); **describe the behavior, not the implementation**.

That said, **use your judgment**: if a detail is genuinely part of the contract and the task can't be pinned down without it, state it. A task nobody can implement is worse than one that names a field.

05

The tests

Your tests are the backbone; the agent only gets the repo and the problem description and is expected to create a solution to pass your tests (they'll be hidden 🤫). They should follow these requirements:

T1 They highlight the missing or incorrect behavior (depending on the task's category: feature request, bug fix, etc). They should 100% fail at the base commit and 100% pass after adding your solution.
T2 The tests should be **deterministic** (no timing, randomness, or ordering; nothing that could change across multiple runs or across different machines).
T3 **Strong tests** 💪. The tests shouldn't be permissive enough to let inaccurate agent solutions pass.
T4 **Extensive coverage.** The tests should cover the requested behavior and all the obvious edge cases.
T5 **DO NOT** check for unspecified or undiscoverable behavior; this will be unfair to the agents to expect them to implement something not in the description or discoverable from the repo 😣.
T6 They don't require any network connection (we run the container with `--network none`).
T7 **Don't over-pin the output** — don't assert on exact output (error text, messages, wording, formatting) unless the description says so or it's obvious from the repo's existing patterns. Otherwise it's unfair to an agent that gets the behavior right but words it differently; checking the behavior holds is enough.
T8 **Keep failure diagnostics intact.** When a test fails, the results should show which test failed and its real assertion output; your framework's default reporting already does this. If you write a custom harness, reporter, or JUnit adapter, it must not hide real failures behind hardcoded catch-all messages, mask upstream errors, or report something different from what actually ran — we rely on those results to judge the runs, so misreporting them is a serious defect.

The `test.sh` harness

You'll create a `test.sh` file at the repo root and include it in your test patch. It has two modes, `base` and `new`:

`./test.sh --output_path results.xml base` — runs the repo's existing tests as a regression check. Must pass. Writes JUnit XML to the output path.

`./test.sh --output_path results.xml new` — runs your new or modified tests. Must fail without the solution patch applied.

If some existing repo tests are flaky, require network, or fail for existing issues, feel free to exclude them. **Do NOT exclude valid tests because your solution breaks them** — we'll know.

- **base runs the real tests** for the area you touched, not a token smoke test; it's a genuine regression check.
- **No fail-fast flags** — we need every test result, not just the first failure.

▶ JUnit XML setup by framework

Generating the test patch

GENERATING A TEST PATCH

```
# Make your test changes, then generate the patch:
git diff > test.patch

# Verify the patch applies cleanly:
git stash
git apply test.patch
git stash pop
```

> **Common mistake: leaking the quest.** Treat these files like a real PR. No directories or files named "challenge", "quest", or "olympus", no `test.sh` comments referencing the challenge, and no "Shipd / Olympus / mars" anywhere in the patches. If it wouldn't show up in a normal PR to the repo, it shouldn't be here.

06

The solution

That's the golden solution: the proof that your task is solvable and that we can pass the tests. The solution should follow these requirements:

S1 The solution should meet all the requirements. (If it's missing a requirement and it manages to pass your tests, you're in a bad position 🤬.)
S2 **No regressions** and **follow existing code patterns**. Don't break existing working code by mistake (we'll still run the repo's existing tests).
S3 **No irrelevant changes** to the code. If something is unrelated to your task, keep it as it is.
S4 **No AI-generated artifacts** (weird comments, unexplained defensive code, new coding patterns, etc).

07

The Dockerfile

We need consistent results, so you'll have to submit a Dockerfile that we'll use to run the agent and tests inside. It should follow these requirements:

- Start `FROM` one of the base images below (pick the one for your language).
- Install all dependencies during the build phase. We run the container with `--network none`, so make sure everything is set up at build time.
- `WORKDIR` must be `/app`. We run tests from there; other paths break imports, editable installs, and relative paths.
- Don't run tests during the build: no test commands in a `RUN` step. The build sets things up; it doesn't test.
- End with `CMD ["/bin/bash"]`
- Works without `test.patch` or `solution.patch` applied; we apply them after the build.

Base images

Pick the one that matches your repo's language:

Python	`public.ecr.aws/d3j8x8q7/olympus-base-python:latest`
TypeScript / JavaScript	`public.ecr.aws/d3j8x8q7/olympus-base-typescript:latest`
Go	`public.ecr.aws/d3j8x8q7/olympus-base-go:latest`
Rust	`public.ecr.aws/d3j8x8q7/olympus-base-rust:latest`
Java (JVM)	`public.ecr.aws/d3j8x8q7/olympus-base-jvm:latest`
C++	`public.ecr.aws/d3j8x8q7/olympus-base-cpp:latest`

Template

A skeleton to start from. Swap `<language>` for `python`, `typescript`, `go`, `rust`, `jvm`, or `cpp`:

DOCKERFILE TEMPLATE

```
FROM public.ecr.aws/d3j8x8q7/olympus-base-<language>:latest

WORKDIR /app

COPY . .

# Install everything you need here — the runtime container is offline.
# Example (Python):
#   RUN pip install -r requirements.txt

CMD ["/bin/bash"]
```

08

Submitting

When you submit on platform, each piece goes in its own field:

- The **problem description** goes in as text.
- The **test patch** is a unified git diff containing your `test.sh` plus the new tests.
- The **solution patch** is a unified git diff with your reference implementation.
- The **Dockerfile** is pasted in directly.

Solvability

At least one agent must solve your challenge before you can submit. If no agent ever passes, your tests are probably too strict, your description is missing something, or the task is unfair. Iterate before you submit.

Legitimate failures will be caught during review and sent back for revision.

Be your own reviewer first

Before you submit, go through your agent runs the way a reviewer will: see how the agents actually behaved against your task. The runs won't catch everything, but the issues they do surface are cheap to fix now and will save you the back-and-forth with reviewers.

> **Are the failures fair?** An agent should fail because the task is genuinely hard — not because a sentence was ambiguous, a requirement was hidden, or a test asked for something the description never stated. If the runs show agents tripping on something you didn't intend, fix it now; after submission it comes back as a revision.
>
> **Are you still clearing the LOC bar?** If agents are solving it in noticeably fewer lines than your solution, your real count is probably lower than it looks. We only count the effective solution lines — the ones that implement the task. Comments, blank lines, generated files, and test code don't count, so don't let them talk you into thinking you're over the bar.

09

Review it locally first

Before you click Submit and spend tokens running checks, I recommend doing exactly what our reviewers do, locally:

01 Clone the repo and check out the exact commit.
02 Apply `test.patch`.
03 Build the Docker image.
04 Run the container with `--network none`.
05 Run `./test.sh base` (must pass) and `./test.sh new` (must fail).
06 Apply `solution.patch`.
07 Rebuild and rerun both modes: both must pass.
08 Confirm no existing PR already solves the issue. (Third time. Still important.)
09 Confirmed your submission passes every point listed in this doc? 👀

VERIFY IT APPLIES CLEANLY

```
git checkout <commit-hash>
git apply test.patch
./test.sh --output_path /tmp/base.xml base   # should pass
./test.sh --output_path /tmp/new.xml new     # should fail
```

10

Submission criteria

Olympus has different submission types, and each has its own bar. Sometimes the bar is lines of code, sometimes it's how many files you touched, sometimes it's how hard the agents had to work to solve the task, sometimes something else entirely.

Rather than restating the current numbers here (they change with product), look at the **"Submission criteria" panel** inside the form when you create your submission. It always shows the exact bar for the assignment you're on.

On LOC specifically: what counts is the effective solution — the lines an agent has to actually write to implement the task and pass the tests. Blank lines, comments, and padding (reordering unrelated code and the like) are excluded, and test code doesn't count at all. Improving your tests doesn't move your LOC, only the solution does. Judge your ideas by the solution they force, not the size of the whole diff.

11

A complete approved example

Here's a recently approved Olympus submission, as the reviewer saw it.

EXAMPLE | GO · FROSTDB APPROVED

DESCRIPTION

frostdb answers grouped aggregation queries by splitting a table into independent partitions, aggregating each partition on its own, and then combining those partial results into one row per group. The aggregations below must return the same answer no matter how the rows of a group are spread across partitions or how many partitions there are.

Add support for distinct aggregations. `count(distinct x)` counts how many different non-null values a group holds, `sum(distinct x)` adds each different value once, and `avg(distinct x)` averages the different values. A value that appears in several rows, or in several partitions, counts only once. `count(distinct x)` accepts numeric or string columns; `sum(distinct x)` and `avg(distinct x)` accept numeric columns. Today a distinct qualifier is ignored, so these behave like their plain counterparts, which is wrong whenever a value repeats.

`· · · trimmed: 4 more aggregation families (variance/stddev, median, geometric/harmonic mean, group_concat) · · ·`

Every one of these aggregations ignores null inputs. When a group has no contributing values the distinct count is zero and the other aggregations are null. All of them are reached through ordinary grouped SQL queries and must coexist with the existing aggregations, which keep working unchanged.

REPO + COMMIT

REPO https://github.com/polarsignals/frostdb
COMMIT 9e5cfe0171adff531d30a9df3e111686996f4a9f

DOCKERFILE

DOCKERFILE

```
FROM public.ecr.aws/d3j8x8q7/olympus-base-go:latest

WORKDIR /app
COPY . .

RUN go mod download

RUN GOBIN=/usr/local/bin go install github.com/jstemmer/go-junit-report/v2@v2.1.0

ENV GOFLAGS="-mod=readonly"
ENV CGO_ENABLED=0

RUN go build ./...

CMD ["/bin/bash"]
```

TEST PATCH

The test patch is a single diff containing both `test.sh` and the new test file.

TEST.SH

```
#!/usr/bin/env bash
# Test runner for the holistic aggregations challenge.
#
# Usage:
#   ./test.sh [--output_path <junit.xml>] <base|new>
#
#   base  run the existing repository tests in the change's blast radius; these
#         must pass both before and after the solution is applied.
#   new   run the new challenge tests; these fail before the solution and pass
#         after it.
set -uo pipefail

cd /app

OUTPUT_PATH=""
if [ "${1:-}" = "--output_path" ]; then
  OUTPUT_PATH="$2"
  shift 2
fi

MODE="${1:-new}"

# A fixed level of parallelism makes the multi-partition behaviour of the query
# engine deterministic regardless of the host's CPU count.
export GOMAXPROCS=4

TEST_LOG="$(mktemp)"
STATUS=0

# run_tests <run-regex> <packages...>
# An empty run-regex runs every test in the given packages.
run_tests() {
  regex="$1"
  shift
  if [ -n "$regex" ]; then
    go test -count=1 -v -run "$regex" "$@" 2>&1 | tee -a "$TEST_LOG"
  else
    go test -count=1 -v "$@" 2>&1 | tee -a "$TEST_LOG"
  fi
  status=${PIPESTATUS[0]}
  if [ "$status" -ne 0 ]; then
    STATUS=$status
  fi
}

case "$MODE" in
  base)
    run_tests "" ./query/ ./query/expr/... ./query/exprpb/... ./query/logicalplan/... ./sqlparse/...
    # query/physicalplan holds two tests that cannot run in a fixed-memory,
    # offline container and are unrelated to this change:
    # Test_Aggregate_ArrayOverflow deliberately allocates more memory than the
    # container has, and Test_Sampler_Randomness is an occasionally flaky
    # statistical test. Run the rest of the package by name.
    run_tests '^(TestAndExprShortCircuits|TestBinaryScalarOperationNotImplemented|TestBuildIndexRanges|TestBuildPhysicalPlan|TestOrderedAggregate|TestOrderedAggregateDynCols|TestOrderedSynchronizer|TestSynchronize|Test_ArrayScalarCompute_Leak|Test_BuildIndexRanges|Test_Sampler|Test_Sampler_Materialize|Test_Sampler_MaxSizeAllocation)$' ./query/physicalplan/...
    ;;
  new)
    run_tests '^TestHA_' ./challenge/...
    ;;
  *)
    echo "unknown mode: $MODE (expected base or new)" >&2
    exit 2
    ;;
esac

if [ -n "$OUTPUT_PATH" ]; then
  go-junit-report -set-exit-code < "$TEST_LOG" > "$OUTPUT_PATH" || true
fi

exit "$STATUS"
```

TESTS ADDED — SNIPPED FOR LENGTH

```
diff --git a/challenge/holistic_aggregation_test.go b/challenge/holistic_aggregation_test.go
new file mode 100644
+++ b/challenge/holistic_aggregation_test.go
@@ -0,0 +1,632 @@
+package challenge
+
+// —— trimmed: imports + arrow record helpers (intSchema, recInt, intByGroup, ...) ——
+
+func TestHA_CountDistinctSingleRecord(t *testing.T) {
+ s := intSchema(t)
+ recs := []arrow.Record{recInt([]string{"a", "a", "a"}, []int64{1, 1, 2}, nil)}
+ got := intByGroup(t, s, recs, "SELECT g, count(distinct value) FROM test GROUP BY g")
+ require.Equal(t, int64(2), got["a"])
+}
+
+func TestHA_CountDistinctAcrossPartitions(t *testing.T) {
+ s := intSchema(t)
+ recs := []arrow.Record{
+   recInt([]string{"a", "a", "a"}, []int64{1, 2, 2}, nil),
+   recInt([]string{"a", "a"}, []int64{2, 3}, nil),
+ }
+ got := intByGroup(t, s, recs, "SELECT g, count(distinct value) FROM test GROUP BY g")
+ require.Equal(t, int64(3), got["a"])
+}
+
+// —— trimmed: 47 more tests across 8 aggregation families (distinct, variance, stddev, median, mean variants, group_concat) ——
```

SOLUTION PATCH

The full solution touches 10 files across `query/logicalplan`, `query/physicalplan`, and `sqlparse`. Here's the first hunk so you can see the shape:

SOLUTION PATCH — SNIPPED FOR LENGTH

```
diff --git a/query/logicalplan/builder.go b/query/logicalplan/builder.go
@@ -231,6 +231,34 @@ func resolveAggregation(plan *LogicalPlan, agg *AggregationFunction) ([]*Aggrega
      Right: countExpr,
    }).Alias(agg.String())

+   return []*AggregationFunction{sum, count}, []Expr{div}, true, err
+ case AggFuncAvgDistinct:
+   sum := &AggregationFunction{
+     Func: AggFuncSumDistinct,
+     Expr: agg.Expr,
+   }
+   count := &AggregationFunction{
+     Func: AggFuncCountDistinct,
+     Expr: agg.Expr,
+   }
+
+   var (
+     countExpr Expr = count
+     aggType   arrow.DataType
+   )
+   aggType, err := agg.Expr.DataType(plan)
+   if !arrow.TypeEqual(aggType, arrow.PrimitiveTypes.Int64) {
+     countExpr = Convert(countExpr, aggType)
+   }
+
+   div := (&BinaryExpr{
+     Left:  sum,
+     Op:    OpDiv,
+     Right: countExpr,
+   }).Alias(agg.String())
+
    return []*AggregationFunction{sum, count}, []Expr{div}, true, err
  default:
    return []*AggregationFunction{agg}, []Expr{agg}, false, nil

// —— trimmed: 9 more changed files (expr.go, validate.go, aggregate.go, distinct/groupconcat/mean/order/variance aggregation files, sqlparse visitor) ——
```

12

Tips

A few things that'll save you tokens and round-trips:

01 **Plan the scope before you start writing patches.**
It's tempting to treat this like a regular open-source contribution: find something to fix or add, write good tests, submit. That can work, but only when the scope is there; if you don't think about the overall scope and the repo's architecture first, it's easy to end up with something too trivial to meet the difficulty criteria. You might be surprised at what the models can already do; the bar moves quickly, and the time to figure out your task is too easy is before you've written 600 lines of solution.

02 **Consider cross-cutting changes.**
A solution that cuts across several layers or subsystems tends to be harder for the agents (and carries more effective LOC) than one confined to a single spot. It's not a requirement, but if your idea keeps landing short on difficulty or LOC, it's one of the better levers you have.

03 **Be smart about spending tokens.**
Running a full batch of agent runs the moment you finish writing is a fast way to drain your balance for nothing. Clear your checks first, then start with a smaller batch to get a vibe: if it's coming back at 100% pass, your task is too easy; if it's 0% on a vague point or an unfair test, you want to catch that before you commit to a full run.

04 **Read the check and agent run outputs.**
Understanding why something passed or failed is half the work. A 0% pass rate isn't always a "bad task" — sometimes it's one ambiguous sentence or one unfair test, and the agent logs are the fastest way to spot it. The more time you spend reading outputs, the faster every later submission goes.

05 **Expect to iterate**